

Q-Commerce Swarm Orchestrator

Minimum Viable Product (MVP) & Technical Requirements Document

1. Executive Summary

The global shift towards Quick Commerce (10-minute grocery delivery) relies heavily on high-density local warehouses known as "Dark Stores." Currently, these facilities face critical bottlenecks during peak order spikes. Unoptimized manual routing leads to picker collisions, misplaced inventory, and delayed dispatches, severely impacting unit economics.

The **Q-Commerce Swarm Orchestrator** is a decentralized AI system designed to solve this multi-billion dollar logistical problem. By combining LLM-based order parsing, Computer Vision anomaly detection, and Reinforcement Learning multi-agent routing, this MVP proves that warehouse picking can be fully optimized into a mathematically synchronized swarm.

2. MVP Scope: "The Single-Store Swarm Sandbox"

The Minimum Viable Product will focus on a heavily constrained, yet deeply complex, simulation of a single high-volume Dark Store. It will encompass the complete end-to-end Machine Learning pipeline and orchestration flow without requiring real-world physical hardware integration.

Core Objective: Prove technical feasibility across Data Engineering, System Architecture, and Machine Learning by reducing simulated picker travel time by a minimum of 40% against a baseline FIFO (First-In, First-Out) model.

3. Target Metrics & Performance

The success of the MVP will be validated against strict, industry-standard machine learning metrics and operational KPIs.

40%+

PICK TIME REDUCTION

> 0.85

NLP BLEU SCORE

> 0.90

CV F1 SCORE

4. Machine Learning & AI Subsystems

A. Natural Language Processing (The Interpreter)

Converts raw, unstructured customer instructions into strictly formatted JSON for the database.

- **Model:** Open-source LLM (e.g., Llama 3 8B) fine-tuned via PEFT/LoRA.
- **Task:** Seq2Seq translation extracting sku_id, quantity, and fragility parameters.
- **Evaluation:** BLEU score calculation to ensure high-fidelity entity extraction without hallucinations.

B. Routing Engine (The Math & Pathing)

Solves the multi-agent Traveling Salesperson Problem (TSP) for high-speed dynamic dispatch.

- **Model:** Reinforcement Learning (Ray RLlib) combined with Google OR-Tools.
- **Task:** Given an array of coordinates, calculate the absolute shortest walking path returning to the dispatch door, utilizing dynamic collision avoidance.

C. Computer Vision (The Eye in the Sky)

A simulated anomaly detection model trained on densely packed retail shelves.

- **Dataset:** Kaggle SKU-110K.
- **Model:** YOLO / Fast-RCNN variant.
- **Task:** Identify misplaced items (e.g., chemicals stored next to food products) and trigger SQL database alerts. Evaluated heavily on F1 Score to balance precision and recall.

D. Recommendation System (Predictive Stocker)

Analyzes purchasing history to propose optimized physical inventory placement.

- **Dataset:** Kaggle Instacart Market Basket Analysis (3M+ rows).
- **Model:** XGBoost / Collaborative Filtering.
- **Task:** Predict item co-occurrence and temporal spikes to recommend overnight physical shelf restructures.

5. Core Technology Stack

LAYER	TECHNOLOGIES	PURPOSE IN MVP
Frontend	Next.js React Framer Motion	Interactive 2D physics simulation of the warehouse grid and picker agent movements.
Backend API	Python FastAPI	High-speed REST API handling order queues and algorithm execution.
Database	PostgreSQL Supabase	Real-time spatial coordinate tracking, inventory ledger, and order state management.
Orchestration	LangGraph	Multi-agent workflow management (Order -> NLP -> Check Stock -> Route -> Dispatch).
ML Platform	PyTorch Vertex AI	Model training, fine-tuning scripts, and metric evaluation pipelines.

6. Relational Database Schema (3NF)

The system relies on a highly normalized PostgreSQL schema to prevent data duplication and ensure ACID compliance during concurrent order processing.

- **locations:** Maps the physical grid. Uses compound constraints (Aisle + Rack + Shelf) and `grid_x`, `grid_y`.
- **skus:** Static product metadata, including a mathematically critical `fragility_score` (1-10) to ensure heavy items are picked before fragile ones.
- **inventory:** The live ledger. A junction table linking SKUs to Locations with strict CHECK (`stock_count >= 0`) constraints.
- **orders:** Tracks state via strict ENUM (`'pending'`, `'routing'`, `'picking'`, `'dispatched'`).
- **order_items:** Maps exact requested quantities to specific SKUs.

7. Execution Plan & Milestones

1. **Phase 1: Foundation (Data & Ledger):** Spin up PostgreSQL schema, seed 50 test SKUs with randomized grid coordinates, and write complex spatial JOIN queries.
2. **Phase 2: The ML Pipeline:** Download Kaggle datasets, set up data cleaning pipelines, and execute base model training for RecSys and NLP fine-tuning.
3. **Phase 3: The Routing Engine:** Implement Google OR-Tools/RLlib to solve the multi-node TSP based on dynamic SQL coordinate lookups.
4. **Phase 4: Orchestration & Frontend:** Connect agents via LangGraph and build the React 2D visual simulation to definitively prove the time reduction.